Implementation

Chapter 10

Introduction

- Start with design
- Implement system as components
  - source code, scripts, object modules, executables, data bases
- Flesh out architecture to build system

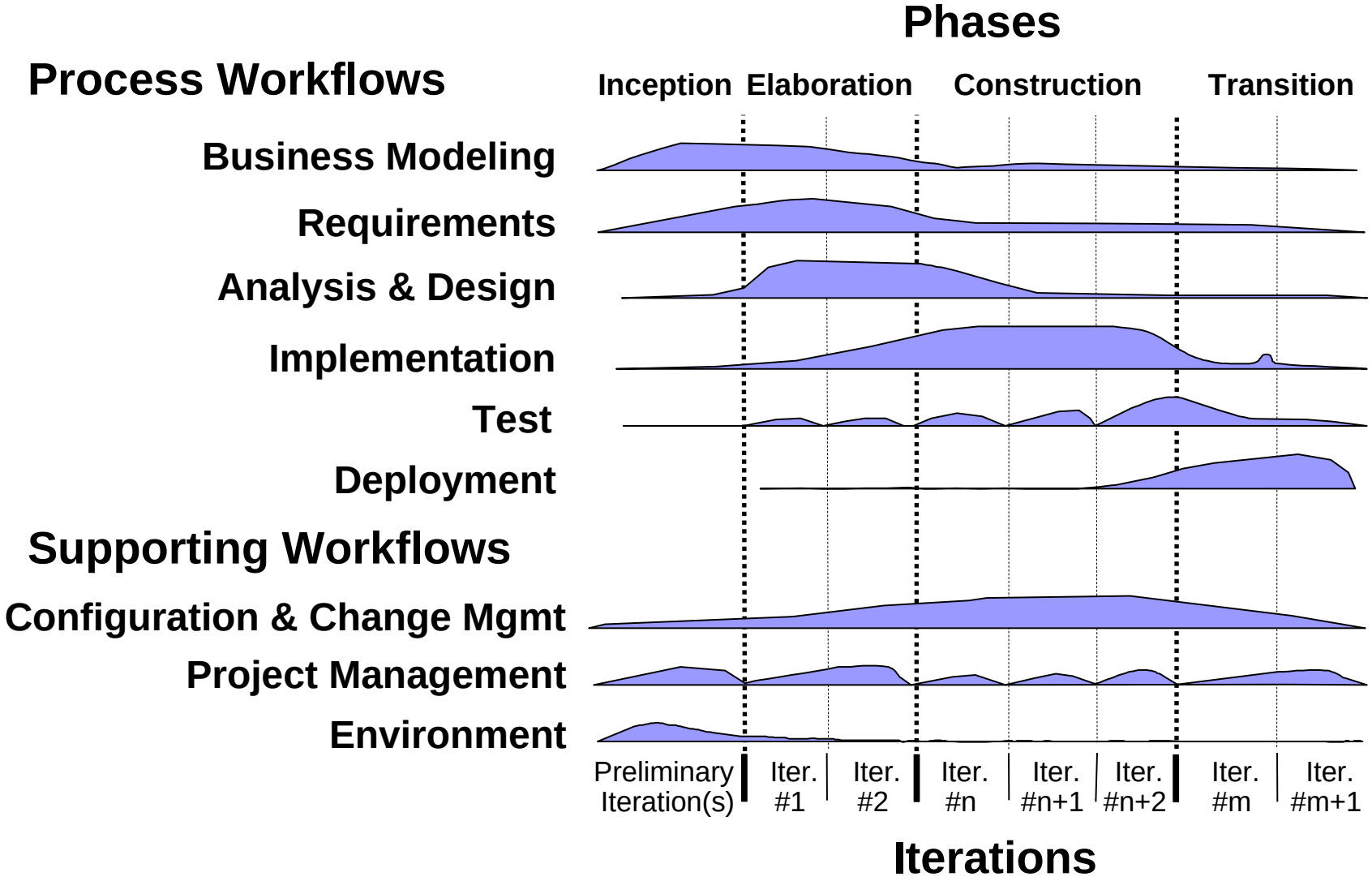
Purpose of Implementation

- Plan system integrations required in each iteration
- Distribute system by mapping executable components onto nodes
- Implement design classes and subsystems
  - create source code
- Unit test components
- Integrate components into executables
  - feeds into test workflow

Key Roles and Artifacts

- Architect responsible for
  - Implementation Model
  - Architecture Description
  - Deployment Model
- Systems Integrator responsible for
  - Integration Build Plan
- Component Engineer responsible for
  - Components
  - Implementation Subsystems
  - Interfaces

Implementation in the Lifecycle



Artifacts

- Implementation Model
- Component
- Implementation Subsystem
- Interface
- Architecture Description
- Integration Build Plan

Implementation Model

- Describes how design model elements are implemented
- Describes how components are organized
- Describes how components depend on each other
- Made up of other artifacts, and shows how they relate to each other

Component

- Physical packaging of model elements
  - such as design classes
- Includes
  - executables
  - file of source code or data
  - libraries of routines (e.g. DLLs)
  - database tables
  - documents
- If represented graphically, can add stereotypes

Traces Between Elements

- Components have trace relationships to the model elements they implement
  - that is, you can identify where an implementation component comes from
- Often a component implements several elements
  - for example, a file could implement several classes
  - a form could contain several controls
- A component can implement an interface

Implementation Subsystem

- Provide the means for organizing implementation artifacts into manageable pieces
- Consists of components, interfaces and other subsystems
- May use language-dependent packaging mechanisms
  - Java or Ada packages
  - VB projects
  - C++ directory of files (--primitive--)

Traces from Design

- Implementation subsystems often trace from design subsystems
  - some of which trace from analysis packages
- The trace is more often 1-1 than with components
  - because subsystems are the architectural building blocks
- Sometimes new low-level subsystems will be discovered during implementation

Interface

- Continue to provide the user view of components or subsystems
- Separation of concerns
  - interface vs. implementation
- Component that realizes an interface must provide all of its operations
- Implementation subsystem that provides an interface must contain components or subsystems that *actually* provide the interface

Architecture Description

- Contains the architectural view of the implementation model
  - depicts the architecturally significant artifacts
- Usually includes
  - decomposition of implementation model into subsystems, their interfaces and dependencies
    - not necessary to include if the same as the design one
  - key components
    - components that trace to significant design classes
    - executable components

Integration Build Plan: Why?

- Software should be built incrementally in manageable steps
- Each step yields a build
  - an executable version of the system
- Each build must be integrated and tested
  - usually put into CM at start of integration
- Each iteration will result in at least one build
  - sometimes several builds needed to get iteration
- Integration Build Plan needed to define this

Integration Build Plan

- Describes sequence of builds required in an iteration
- For each build:
  - defines functionality expected to be implemented in the build
    - listing of use cases and/or scenarios or parts of them
    - can also refer to nonfunctional requirements
  - defines which parts of the implementation model are affected by the build
    - listing of subsystems and components to be implemented

Roles

- Architect
- Component Engineer
- Systems Integrator
- Code Reviewer
  - not mentioned in text, but important role (present in RUP)

Architect

- Responsible for integrity of implementation model
- Ensures model is correct, consistent and readable
- Ensures implementation is consistent with the design architecture
- Responsible for mapping of components onto nodes
- Not responsible for developing components

Component Engineer

- Defines and maintains source code for one or several components
  - makes sure they satisfy design and requirements
- May be responsible for one or more implementation subsystems
  - in this case, often responsible for components contained in subsystem
  - often responsible for design of these subsystems and components

Systems Integrator

- Plans the sequence of builds required for each iteration
- Integrates each build when its parts are ready
  - integration involves testing and diagnosing problems
- When problems are detected, may send components back to the component engineers to fix them
- Same person may also be integration tester (see next module)

Code Reviewer

- As discussed earlier, inspections can be more effective that testing in finding defects
- Project may (should?) schedule reviews of the artifacts when they are complete
- The artifacts that take the most review in this work flow is the code
- The reviewer should follow the procedures (inspection, walkthroughs, code reviews) established on the project

Workflows

- Main goal of implementation is to implement the system
  - Architectural Implementation outlines key components and subsystems
  - Integrate System plans the functionality required in builds and integrates them after implementation
  - Implement a Subsystem for required subsystems
  - Implement a Class brings the design classes to life
  - Perform Unit Test makes sure it works at that level

Integrate System

- Two main activities
  - create an integration build plan at the beginning of the iteration
    - what builds are required?
    - what requirements are implemented in each build?
  - Integrate the builds after the code is ready to get it ready for integration tests
    - get the right versions of the components and subsystems
    - compile and link them to create a build

Implement Subsystem

- Ensure that subsystem has the functionality required by the integration build plan
- This requires the components in a system to be capable of satisfying the use case scenarios
  - if system contains subsystems then some requirements may be delegated to them
- Each class in design subsystem required in current build should be implemented
- Each interface required should be provided

Implement a Class

- At this point, it is a “simple matter of technique”
- Just code the thing and make sure it satisfies the design and provides the right interfaces
  - decide what to put in each source file (if needed)
  - there may be remaining decisions on how to implement associations and aggregations
  - algorithms may need to be chosen and designed
  - data structures may need to be designed
  - internal operations and data may be discovered

Perform Unit Test

- This workflow tests the implemented components as individual units
- The unit's externally observable behaviour should be tested
  - does it give proper output for all inputs?
  - does it handle error cases correctly?
- As well, the structure of the unit should be tested
  - has every line of code been tested, every branch?

Black Box Vs. White Box

- Black box testing of system
  - stimulate it with inputs and evaluate its outputs
  - tester has no access to system internals
  - also called "functional testing"
- White box testing
  - testing of a system by peering into its internals
  - instrumentation is often used
  - sometimes called "structural testing"
- All tests are either black box or white box

Black Box Testing

- Usually used for acceptance tests
  - shows system can work on its own
- Full test requires trying every possible input
- This is not possible
  - use a subset of tests to demonstrate that the system would likely work with all possible inputs
  - e.g., test ranges of inputs, boundary conditions, and selected exception conditions

White Box Testing

- Used for integration tests and unit tests
- In these you don't have a full system to test
  - often need stubs and drivers
- Stubs and drivers are written with a knowledge of the internals of the software
  - exercise all features of the software
- Other types of test monitors and probes are also categorized as 'white box' testing

Test Levels

- Unit testing
- Integration testing
- Acceptance testing
- Less common:
  - simulation testing - during RA, PD and DD phases

Unit Testing

- Done by the individual programmer
  - before releasing a module to the integration team
- White box
- Often driven by the debugger
- Path or branch testing often possible at the unit level
  - see below
- May be too small to test whole use cases

Integration Testing

- Done during development at integration periods
- Tests sets of units together
- Exercises the interfaces between units
- Usually white box
  - generally only testing a subset of the system
  - often need stubs and drivers to do that
- Heavily feature oriented
  - often use case based
  - don't necessarily emulate operational scenarios

Acceptance Testing

- Final stage of testing before delivery of the software to the customer
- Usually 'black box' in nature
- Traces to all requirements use cases
  - all requirements are represented in some test
  - every test verifies one or more requirements
- Usually have the whole system together for acceptance testing

Kinds of Testing

- Many kinds of testing for verifying different kinds of requirements
  - most covered in next chapter
- Two of them may be useful during unit testing
  - function testing
  - performance testing

Function Testing

- Used to test the functional requirements of a system
  - use cases or portions of use cases
- Most commonly used kind of testing
- Tests the normal operation of each function
- Also tests boundary and exception conditions of each function
  - always assume user input may contain errors!!!

Performance Testing

- Tests timing and sizing requirements of system
  - the system can respond to certain stimuli within specified time frames
  - the system can handle a specified volume of data within a given timeframe
  - the system can perform certain functions with a specified periodicity
- Performance requirements should be explicitly written down
  - if not, try to figure what is reasonable

Possible Test Coverage

- Every possible single input tested
- Every combination of inputs tested
- Every path through the software
- Every branch taken at least once
- Every line of code executed
- Every module executed at least once
- Every function executed at least once

- Which are realistic?

Unit Testing Coverage

- Branch testing exercises every possible ’branch' in the software
  - takes every choice at each branch
  - executes every line of code
- Select inputs to drive the code through all possible branches
- Done during unit testing
  - failing this, at least execute every line of code

Unit Testing Coverage

- Tester identifies which data combinations will exercise each choice at each branch
- Tests ensure that these data combinations are used
- Usually requires instrumentation
  - many branches are there to handle exceptions
  - can only be simulated by forcing error conditions

More on Branch Testing

- Path testing is impossible, line of code coverage is too weak
- Branch testing cases cause each choice at each branch code to be taken at least once
  - every **if ... then ... else ...** statement
  - all alternatives of **case** or **select** statements
  - with loops, such as **while** loops, need to take into account the null case

Example of Branch Testing

```
Procedure XXX (A, B, X : integer);
begin
    if ((A > 1) and (B = 0)) then
      X = X/A;
    endif;
    if ((A = 2) or (X > 1) then
      X = X + 1;
    endif;
end XXX;
```

- Every line is executed by the test case (1) A=2, B=0, and X=3
- Also need the following test case: (2) A=1, B=0, X=1, executes neither **then** clause

Discussion of Example

- Test case selection depends on 'white box' knowledge
  - actual test could be black box
- Branch coverage does not cover every path. Two more test cases for that:
  - (3) A=1, B=0, X=2, executes second, not first, and
  - (4) A=4, B=0, X=2, executes first, not second
- Will miss errors
  - occurring in paths that aren't taken
  - where the output value of one condition affects the next one
  - that occur after N iterations of a loop, if the loop isn't done that many times in the test case

Summary

- Primary result of implementation is the implementation model
  - “model” here contains implementation artifacts such as code, forms, databases, etc.
- Includes implementation subsystems
  - with their dependencies, interfaces and contents
- Includes components
  - source and executable files, their dependencies
  - components unit tested
- Includes architectural view of the above